Machine Learning for Finance

Supervised Learning - NonLinear Regression

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

Case Study: Predicting Cross-Sectional Returns

Suppose we want to predict asset returns using firm characteristics.
Target variable:

$$r_i$$

Feature vector:

$$X_i = (X_{1i}, X_{2i}, \ldots, X_{Ki})$$

Goal:

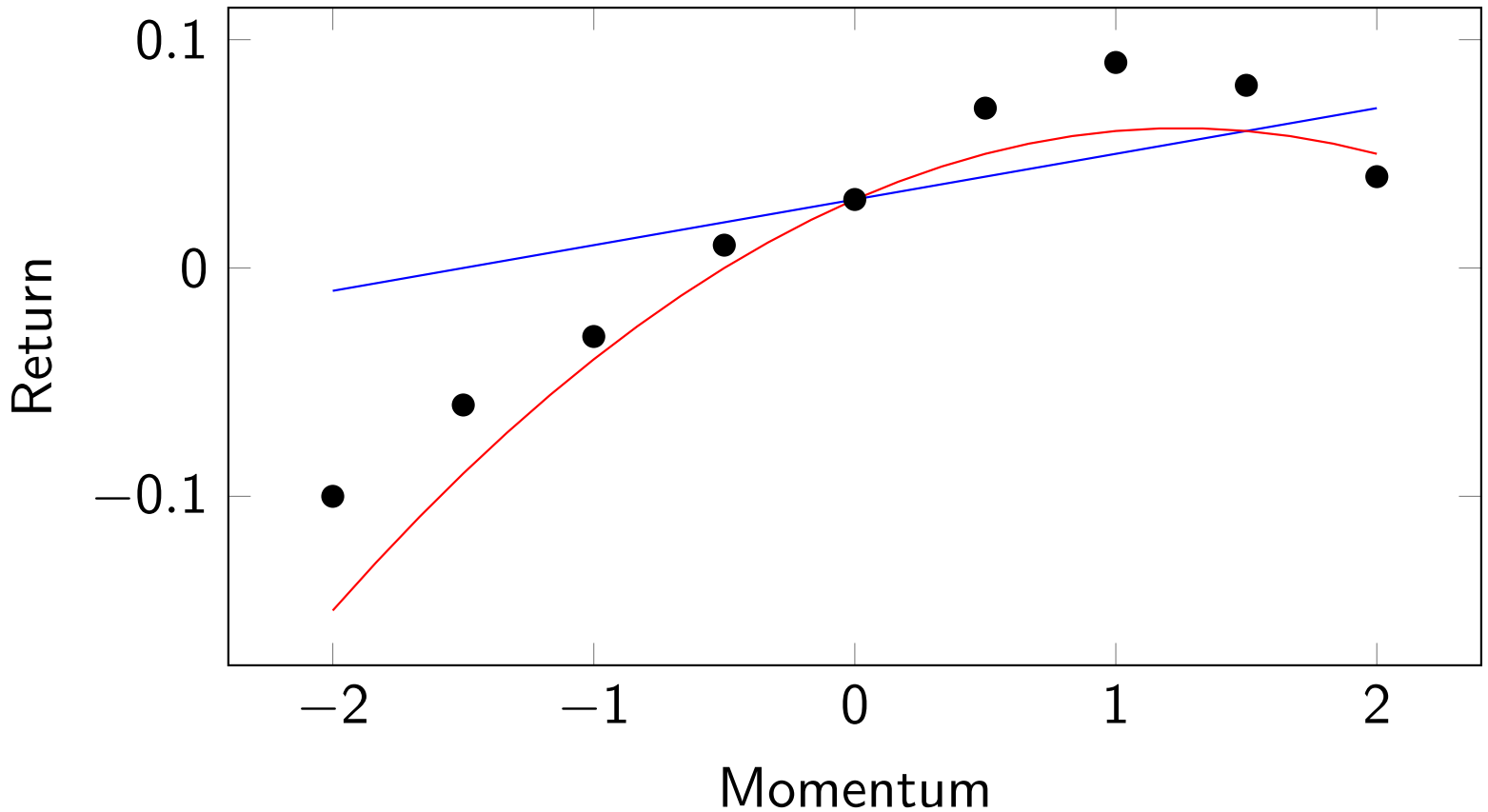
$$r_i = f(X_i)$$

A common assumption is that the relationship is linear:

$$r_i = a + bX_{1i} + cX_{2i}$$

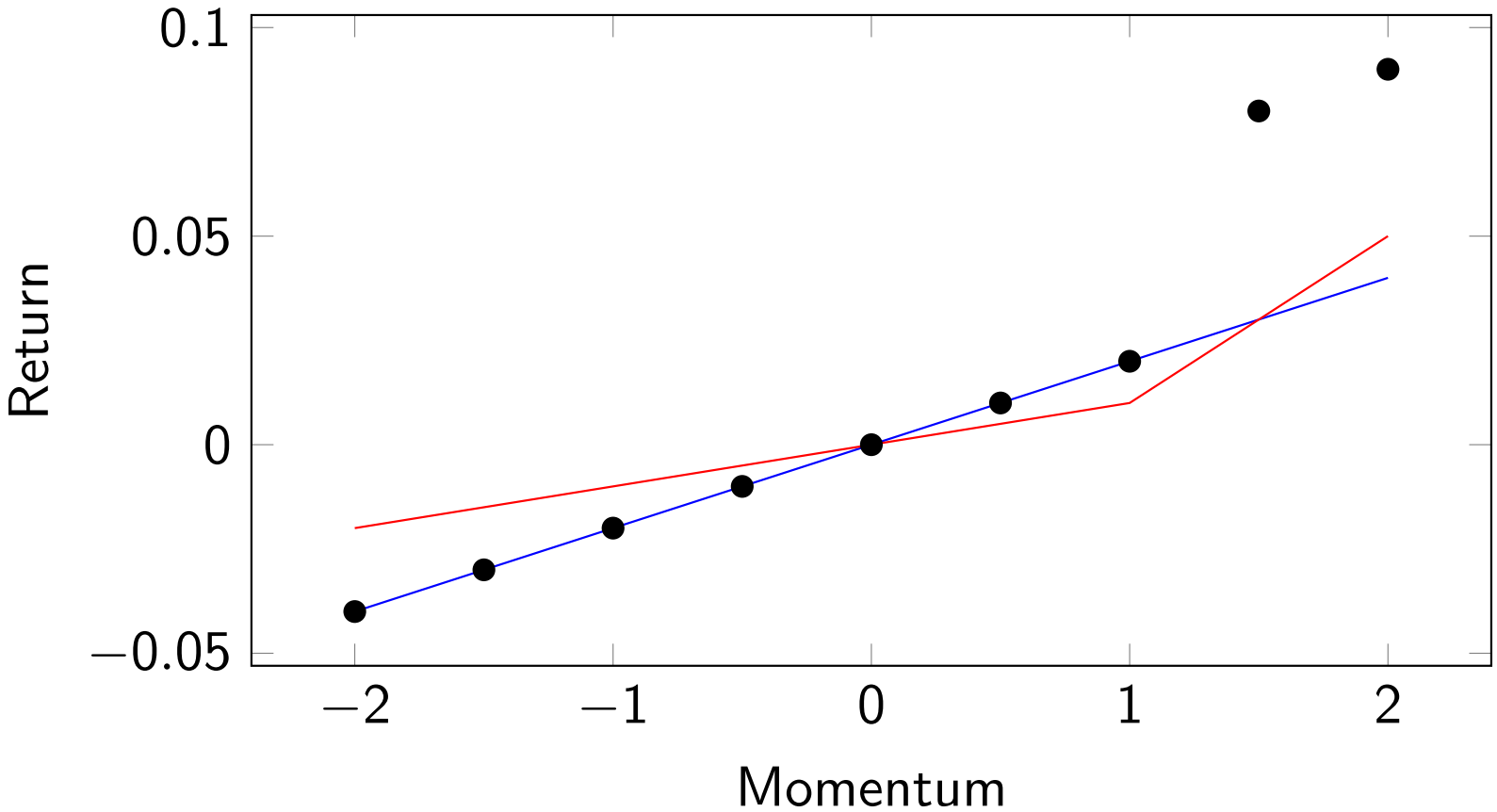
Exploratory Data Analysis

Suppose exploratory data analysis reveals the following pattern.



The linear model may not capture the nonlinear relationship.

Another Possible Nonlinearity



Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

From Linear Regression to GAM

Recall the linear regression model

$$r_i = \alpha + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_K X_{Ki} + u_i$$

This assumes each variable affects returns linearly.
But exploratory data analysis may reveal nonlinear relationships.
A more flexible model allows each variable to enter through an unknown nonlinear function:

$$r_i =$$

This is called a **Generalized Additive Model (GAM)**.

Example: Piecewise Regression

Suppose the relationship between returns and momentum changes at a threshold.
Let the knot occur at $X = 0$.
We can write the model

$$r_i = \alpha + \beta_1 X_i + \beta_2 (X_i - 0)_+ + u_i$$

where

$$(X_i - 0)_+ = \begin{cases} 0 & X_i \leq 0 \\ X_i - 0 & X_i > 0 \end{cases}$$

This allows the slope to change after the knot.
Q: How would a typical OLS approach model this special case of varying slopes?
A:

Spline Representation

Each nonlinear function can be written as a weighted sum of basis functions.

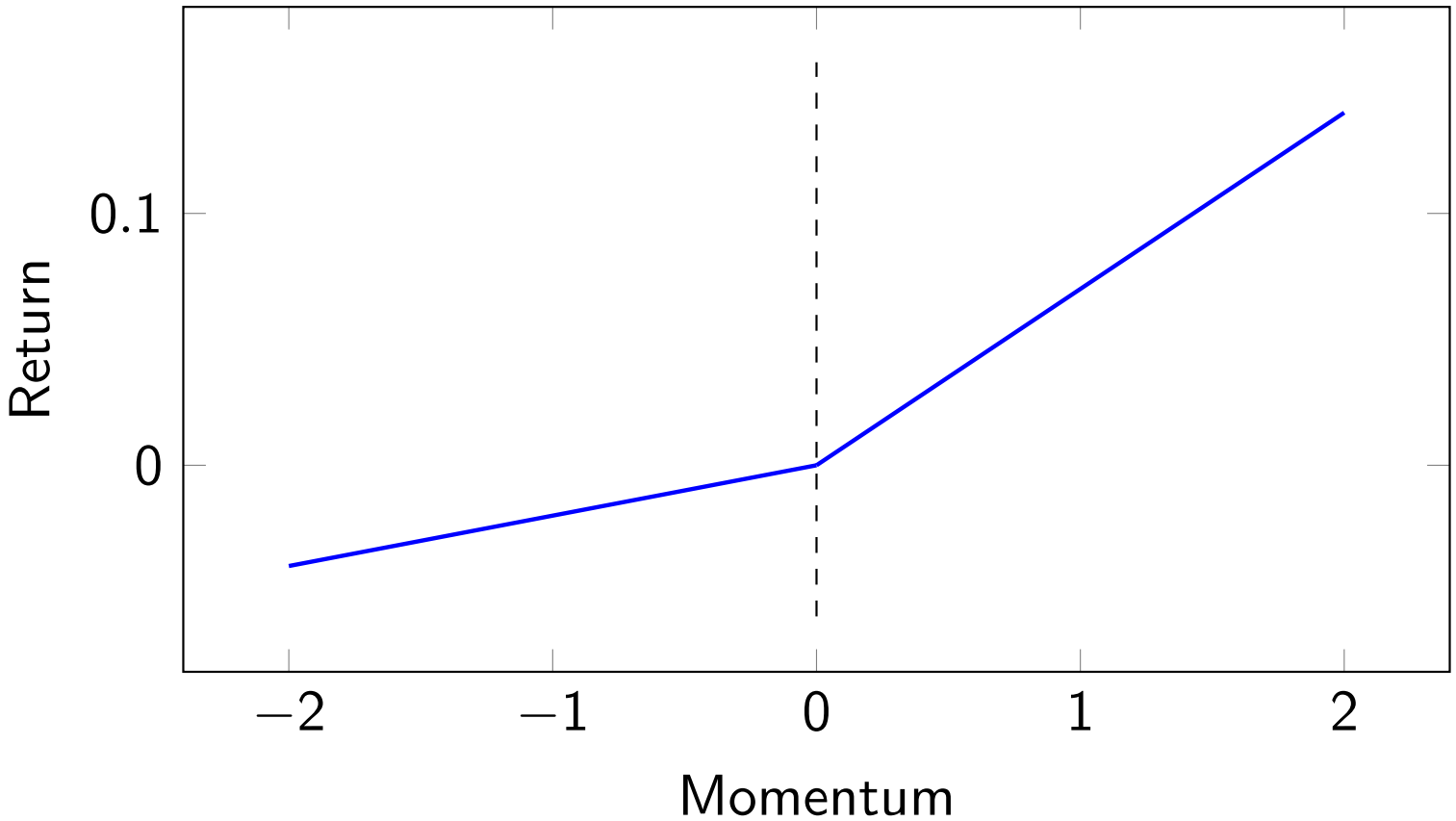
$$f_k(X_k) = \theta_{k1} B_{k1}(X_k) + \theta_{k2} B_{k2}(X_k) + \cdots + \theta_{km} B_{km}(X_k)$$

where

- $B_{kj}(X_k)$ are spline basis functions
- θ_{kj} are parameters to estimate

Combining these basis functions produces a smooth nonlinear curve.

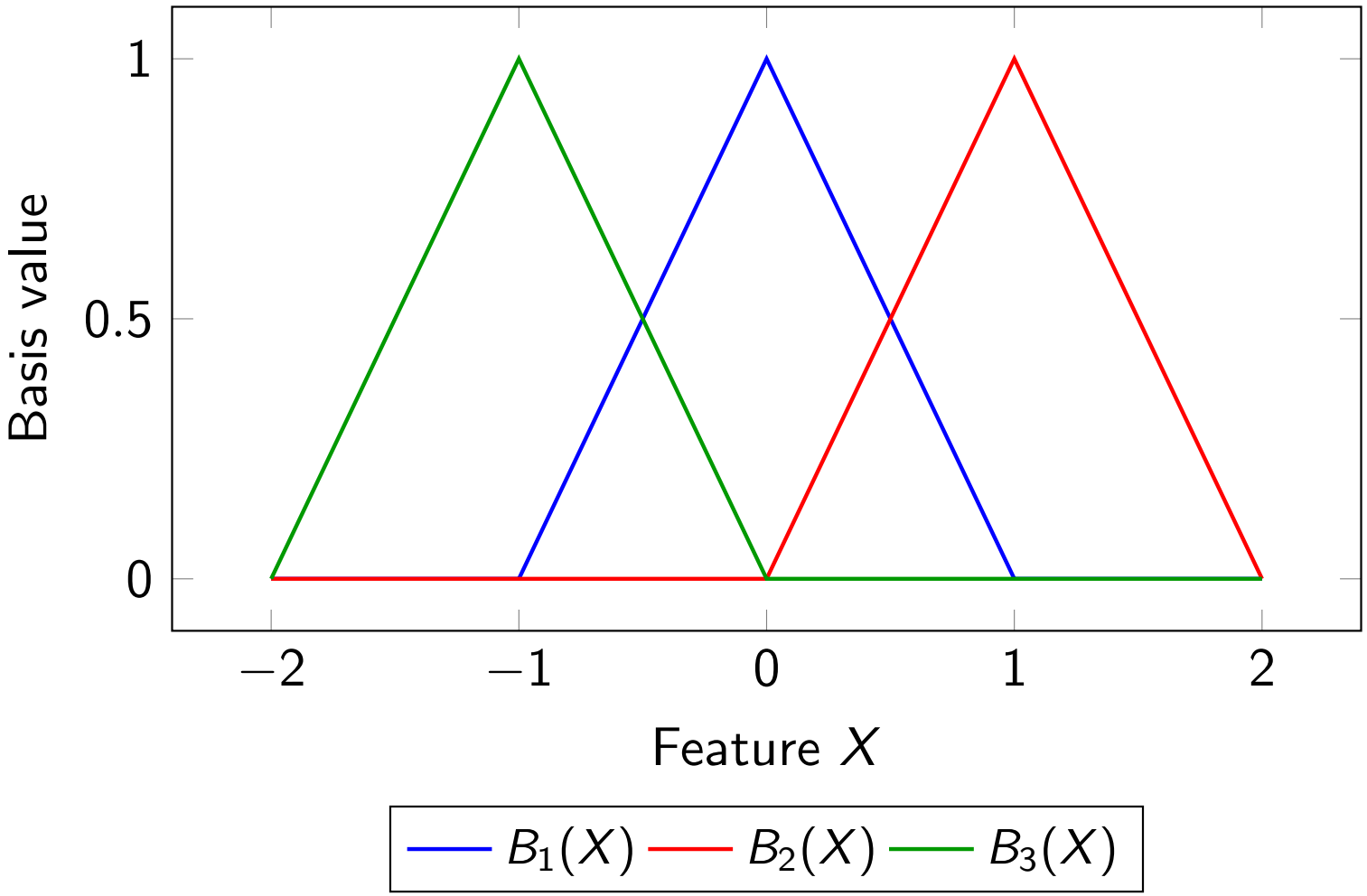
Spline Intuition



Splines allow the relationship to bend at selected knots.

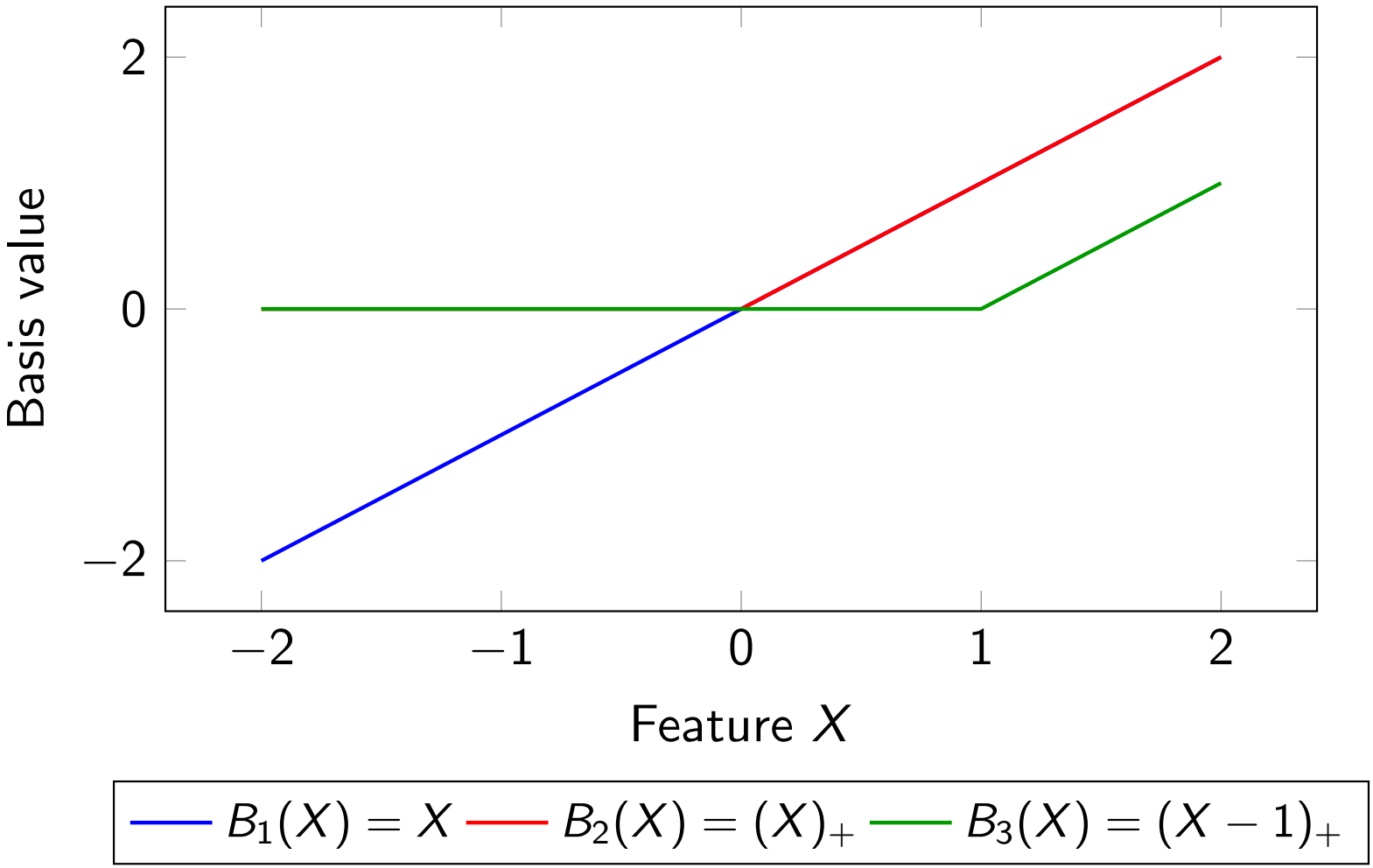
Spline Basis Functions

A spline is constructed from several basis functions.



Each basis function captures

Example of other Basis Functions



These basis functions combine to form flexible nonlinear shapes.

Estimating a GAM

Substituting the spline expansion into the model gives

$$r_i = \alpha + \sum_{k=1}^{K} \sum_{j=1}^{m} \theta_{kj} B_{kj}(X_{ki}) + u_i$$

This is a linear regression in the parameters θ_{kj}.
Parameters are estimated using least squares:
A smoothing penalty is often added to avoid overfitting.

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

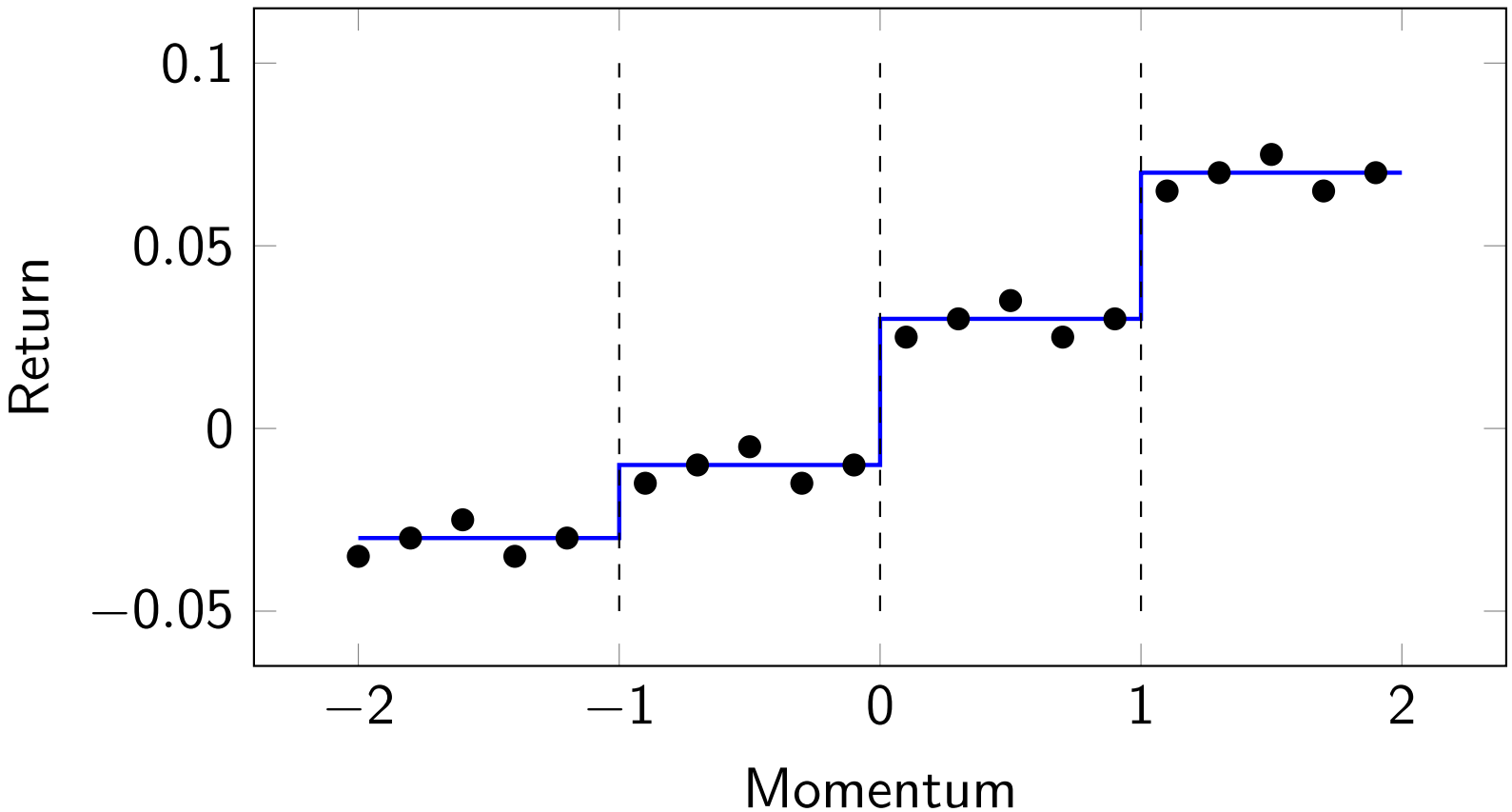
Regression Trees

Recall that decision trees partition the feature space.
For classification we predicted the majority class in each region.
For regression we predict the mean outcome in each region.

$$\hat{r}_i = \text{average return in the leaf node}$$

Trees can therefore approximate nonlinear relationships.

Tree Approximation



Weaknesses of a Single Tree

Single trees have several limitations.

-
- 5*Deep trees can fit noise in the data.*
-

Solution:

Random Forest Idea

Algorithm:
For $m = 1, \ldots, M$:

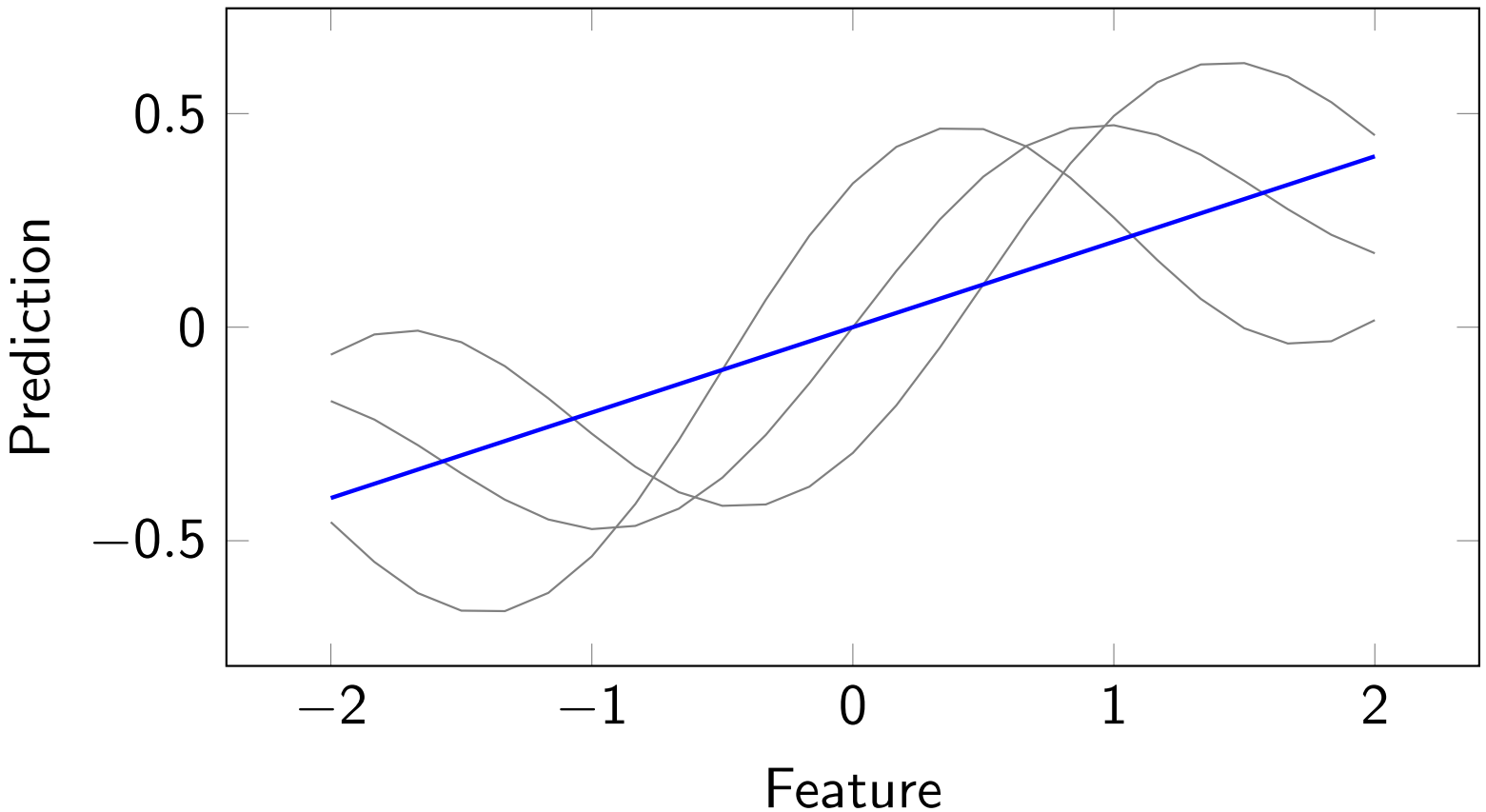
1. Draw a bootstrap sample $\mathcal{N}^{(m)}$
2. Start growing a regression tree on $\mathcal{N}^{(m)}$
3. At each split, consider only a random subset of features
4. Continue until tree T_m is fully grown

Prediction:

$$\hat{f}(x) = \frac{1}{M} \sum_{m=1}^{M} T_m(x)$$

where $T_m(x)$ is the prediction from tree m at feature vector x.

Variance Reduction

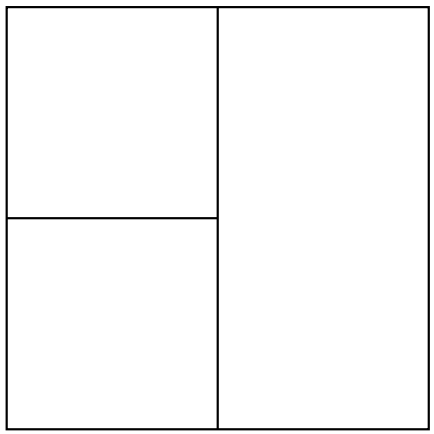


Random Forest Hyperparameters

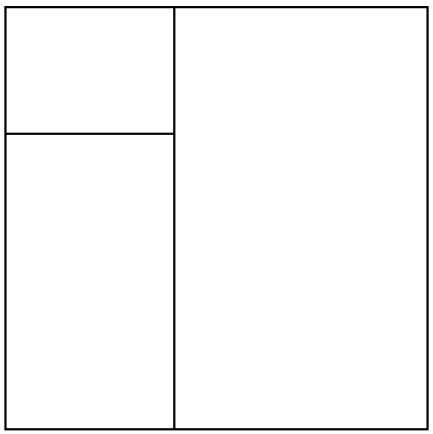
Important tuning parameters:

- Number of trees
- Maximum depth
- Minimum samples per leaf
- Number of features considered per split

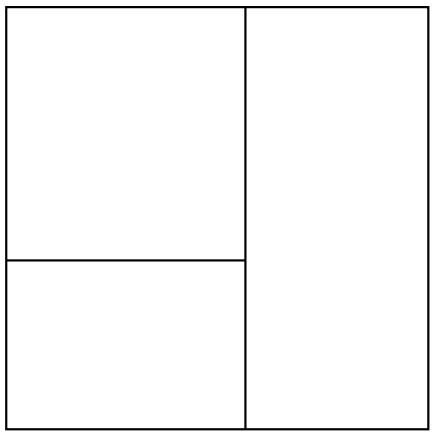
Different Trees from Different Samples



Tree 1



Tree 2



Tree 3

Each tree is trained on a different bootstrap sample.
Small changes in the data produce different tree structures.
Random forests average predictions across many such trees.

Outline

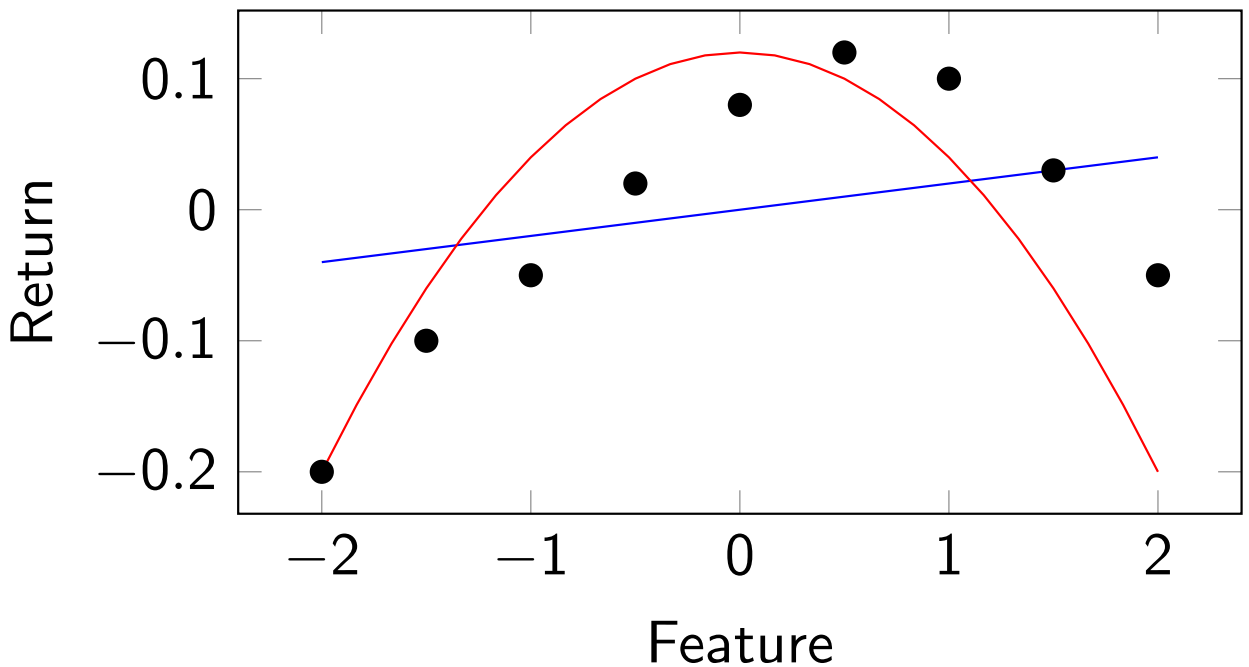
1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

Motivation

Suppose we want to predict returns using firm characteristics.
Linear regression assumes

$$r_i = \alpha + \beta^\top X_i + u_i$$

But relationships between features and returns may be nonlinear.



Neural networks allow us to learn flexible nonlinear functions.

Nonlinear Regression in Econometrics

We already know how to handle nonlinear relationships.
For example, in Generalized Additive Models (GAM):

$$r_i = \alpha + f_1(X_{1i}) + f_2(X_{2i}) + \cdots + f_K(X_{Ki}) + u_i$$

Each function is built using basis expansions:

$$f_k(X_k) = \sum_j \theta_{kj} B_{kj}(X_k)$$

Key idea:

- We approximate nonlinear relationships using transformed features

From GAM to Neural Networks

Neural networks use the same idea.
But instead of choosing basis functions $B_j(X)$ (e.g., splines), they

$$h_j = \sigma(w_j^\top X + b_j)$$

Prediction:

$$\hat{r} = \sum_j v_j h_j$$

where v_j are weights on the hidden units.

- GAM: fixed basis functions
- Neural networks: learned basis functions

Linear Regression as a Special Case

Linear regression:

$$E[r_i] = \alpha + \beta^\top X_i$$

In neural network notation:

$$E[r_i] = w^\top X_i + b$$

Mapping:

$$\beta \leftrightarrow w, \quad \alpha \leftrightarrow b$$

This is a neural network with

-
-

Neural networks generalize this by adding nonlinearities and hidden layers.

Neural Network Structure

We want to predict the target variable r_i

- **Input layer:** features X_i
- **Hidden layers:** intermediate transformations of X_i
- A **unit** (or neuron) computes:

$$h_j = \sigma(w_j^\top X_i + b_j)$$

- The activation function $\sigma(\cdot)$ transforms inputs into nonlinear features
- **Width:** number of units per layer
- **Depth:** number of hidden layers
- **Output layer:** prediction $\hat{r}_i$

Hidden Units

Each hidden unit computes

$$h_j = \sigma(w_j^\top X + b_j)$$

where

- w_j are weights
- b_j is a bias
- $\sigma(\cdot)$ is an activation function

The output combines these features

$$\hat{r} = v_1 h_1 + v_2 h_2 + \cdots + v_m h_m$$

Neural Network Objective

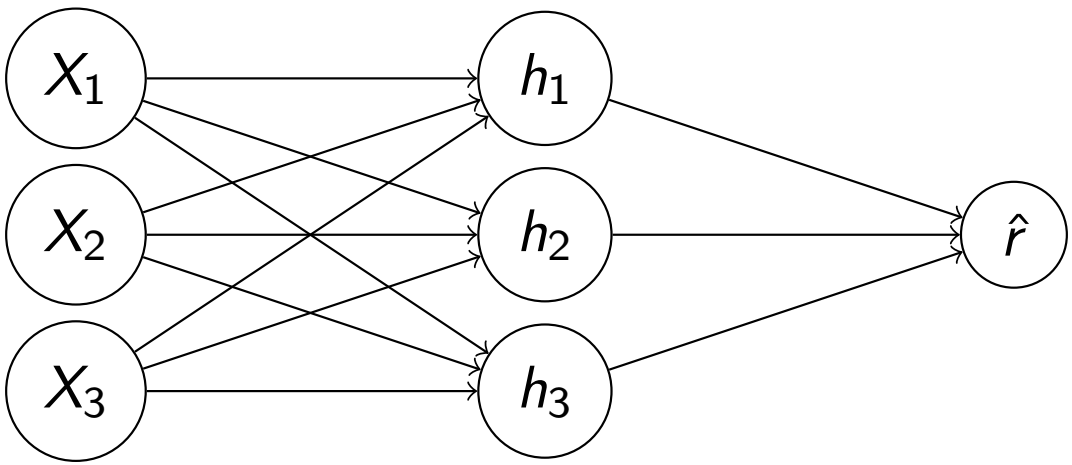
We estimate the network by minimizing prediction error.

$$\min_{\theta} \sum_{i} (r_i - f(X_i; \theta))^2$$

This is the same objective as OLS.
Key difference:

- OLS:
- Neural network:

Single Hidden Layer Network



Hidden units learn nonlinear transformations of the inputs.

Activation Functions

Activation functions introduce nonlinearities.
Common choices:

- ReLU

$$\sigma(x) = \max(0, x)$$

- Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

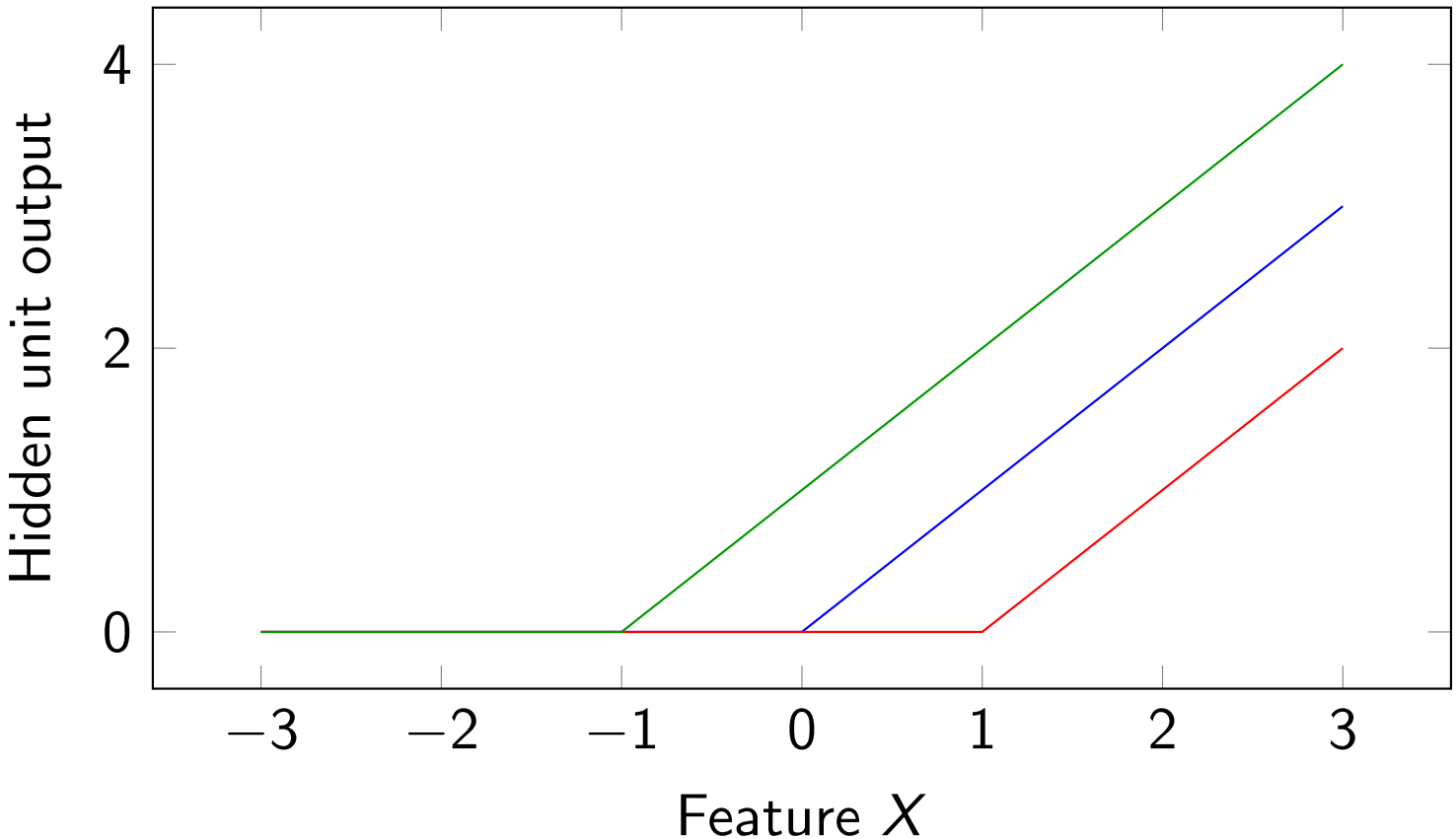
- Tanh

$$\sigma(x) = \tanh(x)$$

Without nonlinear activation functions, a neural network collapses to linear regression.

Hidden Unit Functions

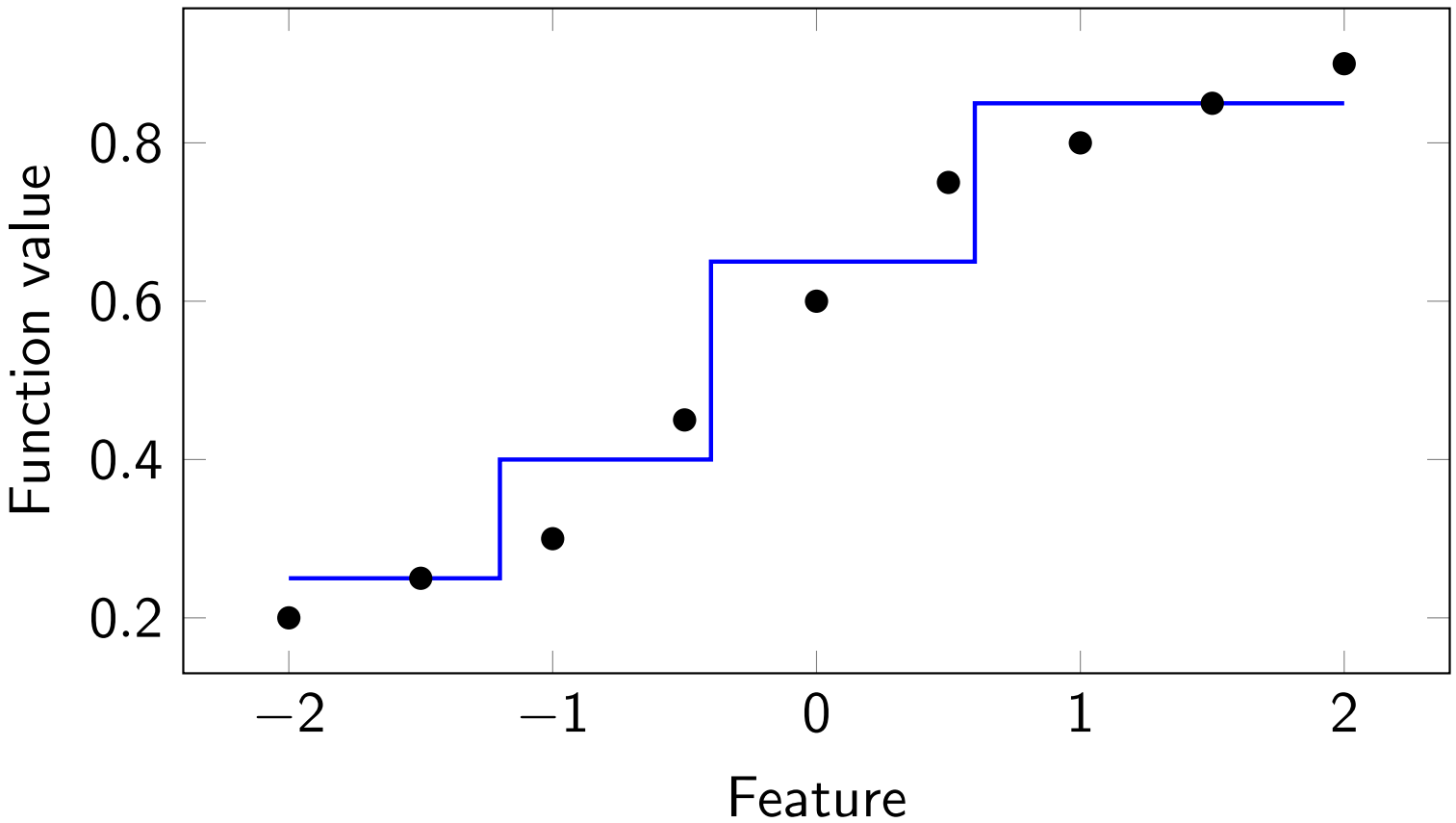
Each hidden unit generates a nonlinear basis function.



These learned functions act as nonlinear features.

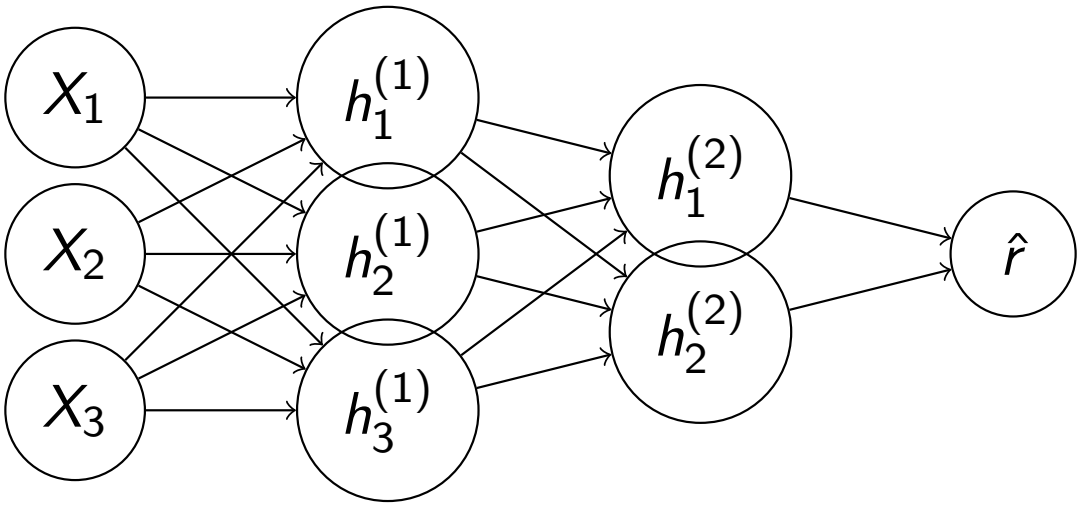
Step Function Approximation

Neural networks can approximate functions by combining many simple units.



With enough hidden units, neural networks can approximate arbitrary functions.

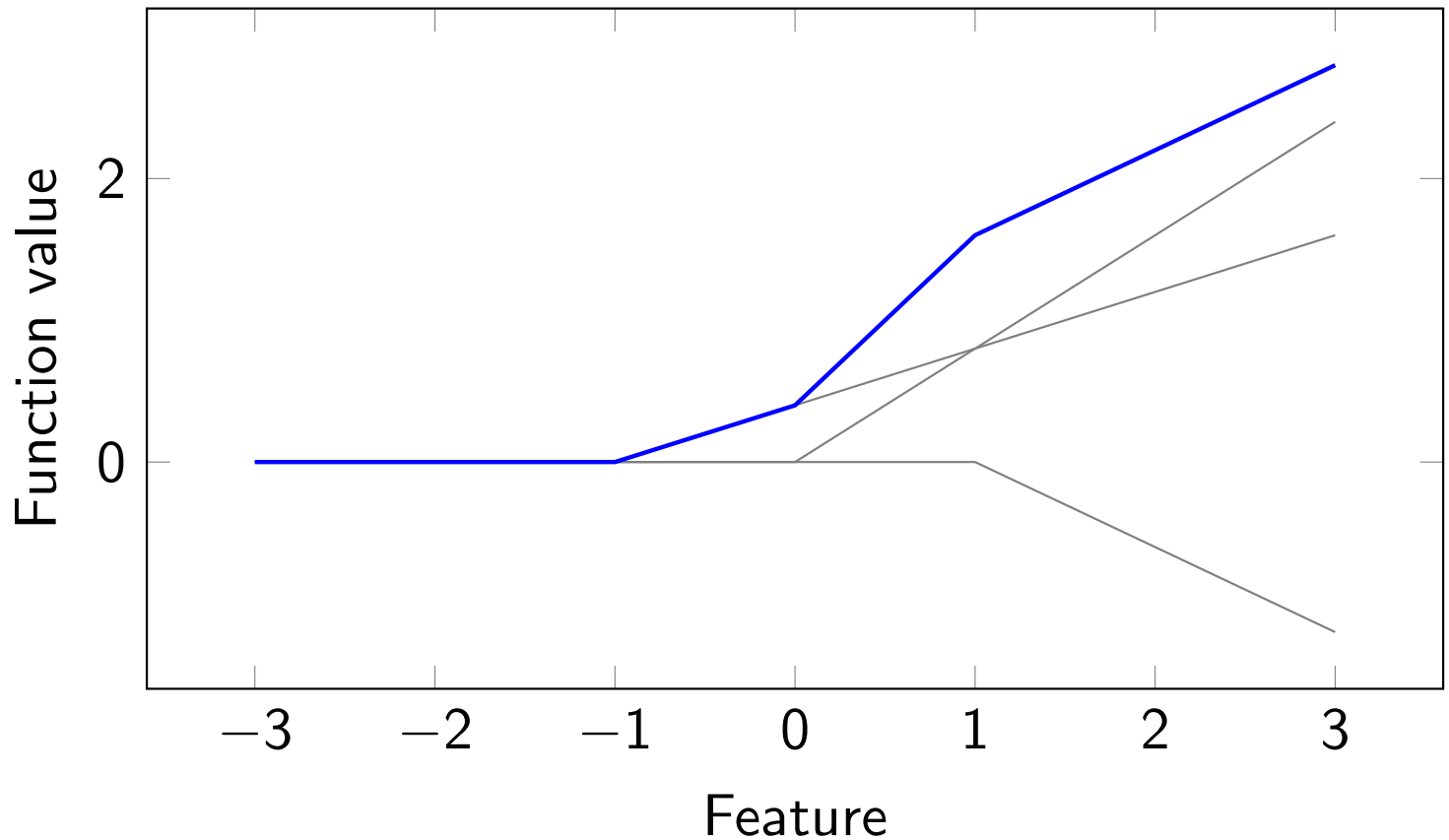
Multi-Layer Neural Network



Multiple layers allow the network to approximate complex functions.

Universal Approximation Intuition

Neural networks approximate nonlinear functions by combining hidden units.



Weighted combinations of hidden units can approximate complex functions.

Neural Network Hyperparameters

Training a neural network requires choosing several hyperparameters.

- **Network width** Number of hidden units per layer.
- **Network depth** Number of hidden layers.
- **Activation function** ReLU, sigmoid, tanh.
- **Regularization** Controls overfitting (weight decay, dropout).
- **Learning rate** Controls the step size during gradient descent.

Hyperparameters are typically chosen using validation data or cross-validation.

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

Motivation

Suppose we want to predict returns.
Start with a simple linear model

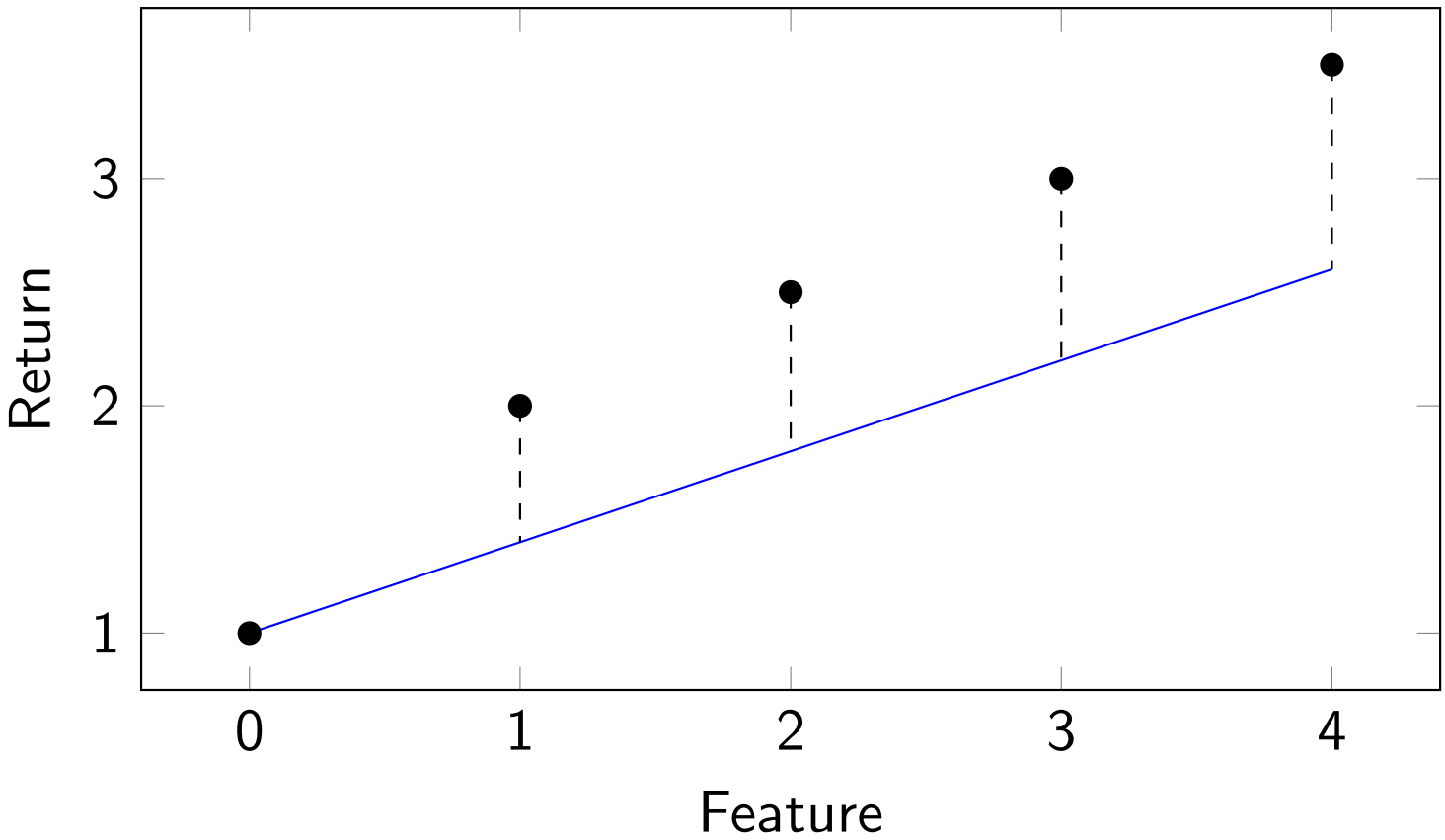
$$r_i = \alpha + \beta X_i + u_i$$

This gives predictions

$$\hat{r}_i^{(1)} = \alpha + \beta X_i$$

But the model may leave systematic errors.

Residuals from the First Model



Residuals represent the remaining errors

$$e_i = r_i - \hat{r}_i^{(1)}$$

Model the Residuals

Residuals may contain predictable structure.
We fit a second model to explain them

$$e_i \approx g(X_i)$$

This model attempts to capture the remaining signal.

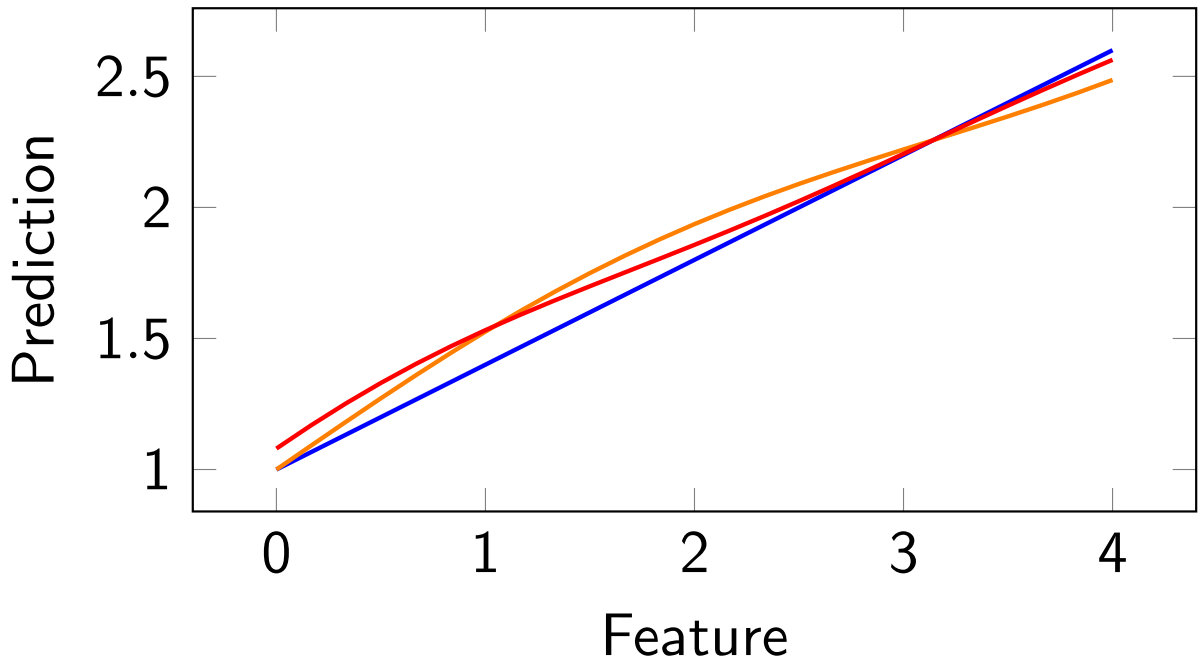
Updating the Prediction

We improve predictions by correcting errors.

$$\hat{r}_i^{(2)} =$$

The second model explains mistakes from the first.

Residual Shrinking Illustration



- Blue: initial model $f_1(x)$
- Orange: updated model after the first correction $f_2(x) = f_1(x) + g_2(x)$
- Red: updated model after another correction $f_3(x) = f_2(x) + g_3(x)$

Boosting improves the fit gradually through small sequential corrections.

Sequential Learning

Boosting repeats the process.

1. Fit model
2. Compute residuals
3. Fit model to residuals
4. Update predictions

Each step improves the overall model.

Boosting Representation

After several iterations the model becomes

$$\hat{r}(x) = f_1(x) + f_2(x) + \cdots + f_M(x)$$

Each model explains remaining errors.
In practice the functions $f_m(x)$ are usually small decision trees.

Gradient Boosting

Gradient boosting formalizes this procedure.
The model evolves as

$$f_m(x) = f_{m-1}(x) + \nu h_m(x)$$

where

- $h_m(x)$ is a regression tree
- ν is a learning rate

Boosting builds a strong model by sequentially correcting errors.

Hyperparameter Tuning

Gradient boosting performance depends on several hyperparameters.

- **Number of trees**
  Controls model complexity.
- **Learning rate**
  Smaller values require more trees but improve stability.
- **Tree depth**
  Controls interaction complexity.
- **Subsampling**
  Using random subsets of data can reduce overfitting.

Hyperparameters are typically chosen using validation data or cross-validation.

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison

Comparing Methods

Method	Key Idea	Flexibility	Interpretability
GAM	Fixed nonlinear basis (splines)	Medium	High
Random Forest	Average many trees	High	Medium
Gradient Boosting	Sequential error correction	Very High	Medium
Neural Network	Learned nonlinear features	Very High	Low

Key takeaway:

- All models estimate the same object: $r_i = f(X_i)$
- They differ mainly in how they approximate $f(\cdot)$

Outline

1. Motivation
2. Generalized Additive Models
3. Random Forests
4. Neural Networks
5. Gradient Boosting
6. Model Comparison